计算机领域本科教育教学改革试点
工作计划（“101计划”）研究成果

数据结构

授课教师：谭光华

湖南大学　信息科学与工程学院

第 4 章

字符串

提纲

4.1　问题引入

4.2　字符串的定义与结构

4.3　字符串的存储实现

4.4　字符串的模式匹配

4.5　拓展延伸

4.6　应用场景

4.1 问题引入：模式匹配

- **查找功能**
  - **文本编辑工具中，给定一段文本，用户提供特定的关键词，找出这个关键词在文本中出现的位置，就是字符串匹配问题。**

输入关键词 → 字符串匹配算法 → 定位关键词位置

  - **用户给定的关键词或字符串被称为模式串，段落文本成为目标串，因此字符串匹配又称为模式匹配。**

4.2 字符串的定义与结构

- **字符串**
  - 数据元素都是**字符**的线性表，记为s=” $s_1s_2...s_n$”
    - s为字符串名，双引号为字符串的定界符，双引号之间的内容是字符串的值
    - s_i（1≤i≤n）可以是字母、数字或其他字符
  - ∅ 表示空串，空串的长度为0
- **子串**
  - 字符串中**任意个连续的字符**组成的子序列
  - 如长度为4的字符串s="abcd"，其子串包括
    - "a"、"ab"、"abc"、"abcd"、"b"、"bc"、"bcd"、"c"、"cd"、"d"、""（空字符串）

字符串的抽象数据类型定义

ADT String {

数据对象： $D=\{ s_i \mid s_i \in \text{CharacterSet}, i=1, 2, \ldots, n, n>0 \}$或$\varnothing$ ，表示空字符串

数据关系： $R=\{ <s_{i-1}, s_i> \mid s_{i-1}, s_i \in D, i=2, \ldots, n, n>0 \}$

基本操作:

InitStr(s)：初始化一个空的字符串s，字符串最大长度为kMaxSize。

StrCopy(s)：返回复制字符串s得到的字符串。

StrIsEmpty(s)：判断字符串s是否是空串。返回一个布尔值，若字符串s是空串，返回true；否则返回false。

StrInsert(s, pos, t)：在字符串s的pos位置处插入字符串t，并返回插入后的字符串s。

StrRemove(s, pos, len)：删除字符串s中从pos位置开始的长度为len的子串，返回删除后的字符串s。

SubString(s, pos, len)：返回字符串s从pos位置开始长度为len的子串。

StrLength(s)：返回字符串s的长度。

StrConcat(s,t)：返回字符串s和t联接而成的新串s。

StrCompare(s,t)：返回字符串s和t的大小关系。若s>t，返回+1；若s=t，返回0；若s<t，返回-1。

PatternMatch(s,t)：返回字符串s中首次出现字符串t的位置，若字符串s没有出现字符串t，则返回NIL。

Replace(s, sub_s, t)：将字符串s中的所有子串sub_s替换为字符串t。}

4.3 字符串的存储实现

字符串存储表示

- **顺序存储结构**：将所有的数据元素存放在一段连续的存储空间中，数据元素的存储位置反应了它们之间的逻辑关系
- **链接存储结构**：逻辑上相邻的数据元素不需要在物理位置上也相邻，数据元素的存储位置可以是任意的

字符串的顺序存储实现

- **字符串的顺序存储结构：即顺序表的存储方式**
  - **用一组地址连续的存储单元来存储串中的字符序列，也是最为常见的字符串存储结构**
  - **字符串"abcdef"的顺序存储形式**

0	1	2	3	4	5	6
a	b	c	d	e	f	\0

    - **注意：一般在程序设计语言中，字符串会有结束符，如C语言和C++语言中字符串的结束符为'\0'，结束符不计入字符串长度，但要占存储空间。**

顺序存储的字符串的基本操作

- 1. 字符串插入
  - 在字符串s的pos位置插入字符串t
    - 首先将字符串s从pos位置开始整体后移，为字符串t的插入提供空间
    - 然后将字符串t中的字符逐一插入到字符串s从pos开始的空间中，得到插入后的字符串s
    - 最后更新字符串s的length属性。
  - 时间复杂度
    - 最坏情况是将t插在s的头，时间复杂度是O(n+m)，
      - 其中n和m分别是两个字符串的长度。

字符串插入

算法4-1 字符串插入操作StrInsert(s, pos, t)

输入：字符串s ，要插入的位置pos≥1，需要插入的字符串t
输出：完成插入后的字符串s。若插入后的字符串长度大于kMaxSize，则直接退出

```
n ← s.length
m ← t.length
if n+m≤kMaxSize then
| for i←n-1 downto pos do
| | s.data[i+m] ← s.data[i]  //将数组下标pos开始的子串后移，给t 留出空位
| end
| for i←0 to m-1 do
| | s.data[pos+i] ← t.data[i]  //将t 插入s
| end
| s.length ← n + m  //更新s的长度
else
| 长度超限，退出
end
```

顺序存储的字符串的基本操作

- **2. 字符串删除**
  - **将字符串s从pos位置开始删除长度为len的子串**
    - **将字符串s中pos+len后面的部分逐位向前移动**
    - **更新字符串s的length属性。**
  - **时间复杂度**
    - **最坏情况是将s开头的len个字符删除，时间复杂度是O(n)，**
      - 其中n是字符串s的长度。

算法4-2 字符串删除操作StrRemove(s, pos, len)

输入：字符串s，要删除的位置pos≥1，删除的字符个数len
输出：删除子串后的字符串s

```
n ← s.length
if pos+len-1<n then
| for i ← pos+len-1 to n-1 do
| | s.data[i-len] ← s.data[i]
| end
| s.length ← n – len
else //从数组下标pos-1开始的所有字符都删掉
| s.length ← pos-1
end
```

顺序存储的字符串的基本操作

- **3. 字符串截取**
  - **返回字符串s从pos位置开始长度为len的子串。**
    - **构造新串，将字符串s从pos之后的len个字符一一赋值给新串**
  - **时间复杂度**
    - **该操作仅与子串长度有关，为O(len)。**

算法4-3 字符串截取子串操作SubString(s, pos, len)

输入：字符串s，开始截取的位置pos≥1，截取的字符个数len
输出：截取的子串

```
InitStr(sub_s)  //初始化新串sub_s
n ← s.length
i ← 0
while pos-1+i<n 且 i<len do   //若s从pos-1开始不到len个字符，就截取到s的末尾为止
|  sub_s.data[i] ← s.data[pos-1+i]  //将s串从pos-1之后的len个字符复制到sub_s
|  sub_s.length ← sub_s.length + 1
|  i ← i+1
end
return sub_s
```

顺序存储的字符串的基本操作

- **4. 字符串连接**
  - **将字符串t连接在字符串s末尾。**
    - **将字符串t中的字符一一赋值到s末尾，并更新s的长度。**
  - **时间复杂度**
    - **该操作仅与t的长度有关，为O(m)。**

算法4-4 字符串连接操作StrConcat(s,t)

输入：字符串s，字符串t
输出：返回字符串s后面联接t而成的新串。若结果长度大于kMaxSize, 则退出。

```
n ← s.length
m ← t.length
if n+m ≤ kMaxSize then
| for i ← 0 to m-1 do
| | s.data[n+i] ← t.data[i]
| end
| s.length ← n+m
else
| 长度超限，退出
end
```

顺序存储的字符串的基本操作

- **5. 字符串比较**
  - **从左到右逐个字符按照其ASCII码值进行比较。**
    - **如果两个字符串长度相等，且每一个相应位置上的字符都相同，则这两个字符串相等，如"abc"与"abc"相等。**
    - **如果两个字符串长度不相等，但较短字符串所有对应位置上的字符都与较长字符串相同，则字符串长度长的字符串大于长度短的字符串。如"abc"<"abcdef"。**
    - **如果两个字符串在某一相应位置上的字符不相同，则以第一个不相同的位置上的字符比较结果作为两个字符串的比较结果。如"abh">"abf"。**
  - **时间复杂度**
    - **仅与s和t的最小长度有关，为O(min(n, m))。**

算法4-5 字符串比较操作StrCompare(s,t)

```
输入：字符串s ，字符串t
输出：s>t输出+1；s=t输出0；s<t输出-1
len ← Min(s.length, t.length)
i ← 0
while i<len 且s.data[i] = t.data[i] do
   i ← i+1
end
if i=len then                          // 所有字符均相同
|  if s.length > len then
|  |  ret ← 1
|  else if t.length > len then
|  |  ret ← -1
|  else  //s=t
|  |  ret ← 0
|  end
else if s.data[i] > t.data[i] then
|  |  ret ← 1
else
|  |  ret ← -1
end
return ret
```

字符串的链接存储实现

- **非紧缩链接存储结构**
  - ✓ **每个结点存放一个字符**

- **紧缩链接存储结构（也称为 块链存储结构）**
  - ✓ **一个结点存放多个字符**

为容易理解，下面对非紧缩链接存储结构的字符串的基本操作进行介绍

链接存储的字

- **1. 字符串插入**
  - **找到字符串s中位于pos位置的链表元素，**
  - **找到插入字符串t的末尾元素**
  - **更改相关指针**
- **时间复杂度**
  - **O(n+m)**
  - **其中n和m分别是两个字符串的长度**

算法4-6 字符串插入操作StrInsert(s, pos, t)

```
输入：字符串s，要插入的位置pos≥1，需要插入的字符串t
输出：完成插入后的字符串s。若不存在位置pos，则s不变
flag ← NormalCode
if t.length>0 then //若t不是空串
  tail ← t.head
  while tail.next ≠ NIL do //找到t的最后一个元素
    tail ← tail.next
  end
  if s.length>0 then //若s不是空串
    p ← s.head
    count ← 1
    while p ≠ NIL 且 count<pos-1 do //找第pos个元素的前一个元素
      count ← count + 1
      p ← p.next
    end
    if count=(pos-1) then //将t插在p的后面
      tail.next ← p.next
      p.next ← t.head
    else if pos = 1 then //t插在s的头
      tail.next ← s.head
      s.head ← t.head
    else
      flag ← ErrorCode //输入错误：不存在位置pos
    end
  else //若s是空串
    s ← t
  end
end
if flag ≠ ErrorCode then  //正常完成插入
  s.length ← s.length + t.length
end
```

链接存储的字

- **2. 字符串删除**
  - **将pos-1位置上的链表元素的next指针指向原字符串s中的第pos+len个元素，并释放被删除的结点空间**

- **时间复杂度**
  - **O(n)**

算法4-7 字符串删除操作StrRemove(s, pos, len)

输入：字符串s ，要删除的位置pos≥1，删除的字符个数len
输出：删除子串后的字符串s。若删除位置不存在，则s不变。

```
if s.length>0 then //若s不是空串
| p ← s.head
| count ← 1
| while p ≠ NIL 且 count<pos-1 do //找第pos个元素的前一个元素
| | count ← count + 1
| | p ← p.next
| end
| if pos=1 或 (p ≠ NIL 且count=pos-1) then //将p后面的len个结点删除
| | if pos=1 then
| | | deleted ← s.head
| | else
| | | deleted ← p.next
| | end
| | count ← 0
| | while deleted≠ NIL 且 count < len do //不足len个则一直删到末尾
| | | t ← deleted.next
| | | delete deleted
| | | count ← count + 1
| | | s.length ← s.length - 1
| | | deleted ← t
| | end
| | if pos = 1 then
| | | s.head ← deleted
| | else
| | | p.next ← deleted
| | end
| end
end
```

链接存储的字

- **3. 字符串截取**
  - **返回字符串s从pos位置开始的长度为len的子串。**
  - **构造新串，将字符串s从pos之后len个字符一一赋值给新串。**
- **时间复杂度**
  - **O(n)**
  - **需要找到起始位置pos，所以不仅与子串长度有关**

算法4-8 字符串截取子串操作SubString(s, pos, len)

```
输入：字符串s，开始截取的位置pos≥1，截取的字符个数len
输出：截取的子串
InitStr(sub_s)  //初始化新串sub_s
if s.length>0 then //若s不是空串
| p ← s.head
| count ← 1
| while p ≠ NIL 且 count<pos do //找第pos个元素
| | count ← count + 1
| | p ← p.next
| end
| if p ≠ NIL 且count=pos then //将s串从pos之后的len个字符复制到sub_s
| | count ← 0
| | sub_s.head ← new StringNode() //创建临时空头结点
| | tail ← sub_s.head
| | while p ≠ NIL 且 count<len do //若从pos开始不到len个字符，就截取到s的末尾
| | | t ← new StringNode(p.data, NIL) //复制一个新结点
| | | tail.next ← t  //将新结点接到sub_s的末尾
| | | tail ← tail.next
| | | sub_s.length ← sub_s.length + 1 //sub_s 长度加1
| | | p ← p.next
| | | count ← count + 1
| | end
| end
end
tail ← sub_s.head
sub_s.head ← tail.next
delete tail  //删除临时空头结点
return sub_s
```

链接存储的字符串的基本操作

- **4. 字符串连接**
  - **找到字符串s末尾并将字符串t接在后面即可。**

- **时间复杂度**
  - **O(n)，找到字符串s末尾的时间开销**

算法4-9 字符串连接操作StrConcat(s,t)

输入：字符串s，字符串t
输出：返回后面联接t而成的字符串s

```
if s.length > 0 then //若s非空串
| p ← s.head
| while p.next ≠ NIL do  //找到s的最后一个结点
| | p ← p.next
| end
| p.next ← t.head
else  //若s是空串
| s.head ← t.head
end
s.length ← s.length + t.length
```

链接存储的字符串的基本操作

- **5. 字符串比较**
  - **返回字符串s和字符串t的大小关系。**
    - **若s>t，返回+1；**
    - **若s=t，返回0；**
    - **若s<t返回-1。**
- **时间复杂度**
  - **O(min(n, m))，仅与s和t的最小长度有关**

算法4-10 字符串比较操作StrCompare(s,t)

```
输入：字符串s ，字符串t
输出：s>t输出+1；s=t输出0；s<t输出-1
        sp ← s.head
        tp ← t.head
        while sp≠NIL 且 tp≠NIL且 sp.data = tp.data do
        | sp ← sp.next
        | p ← tp.next
        end
        if sp≠NIL且tp=NILthen
        | ret ← 1
        else if sp=NIL且tp≠NIL then
        | ret ← -1
        else if sp=NIL且tp=NIL then //s=t
        | ret ← 0
        else if sp.data > tp.data then
        | | ret ← 1
        else //sp.data < tp.data
        | | ret ← -1
        end
        return ret
```

4.4 字符串的模式匹配

- **模式匹配**
  - 在字符串s中找出与字符串t相等的子串，又称子串的定位
  - 字符串s称为**主串或目标串**
  - 字符串t称为**模式串**
- **模式匹配算法**
  - 朴素字符串匹配算法（BF算法）
  - KMP算法
  - BM算法、KR算法、Sunday算法

朴素模式匹配算法

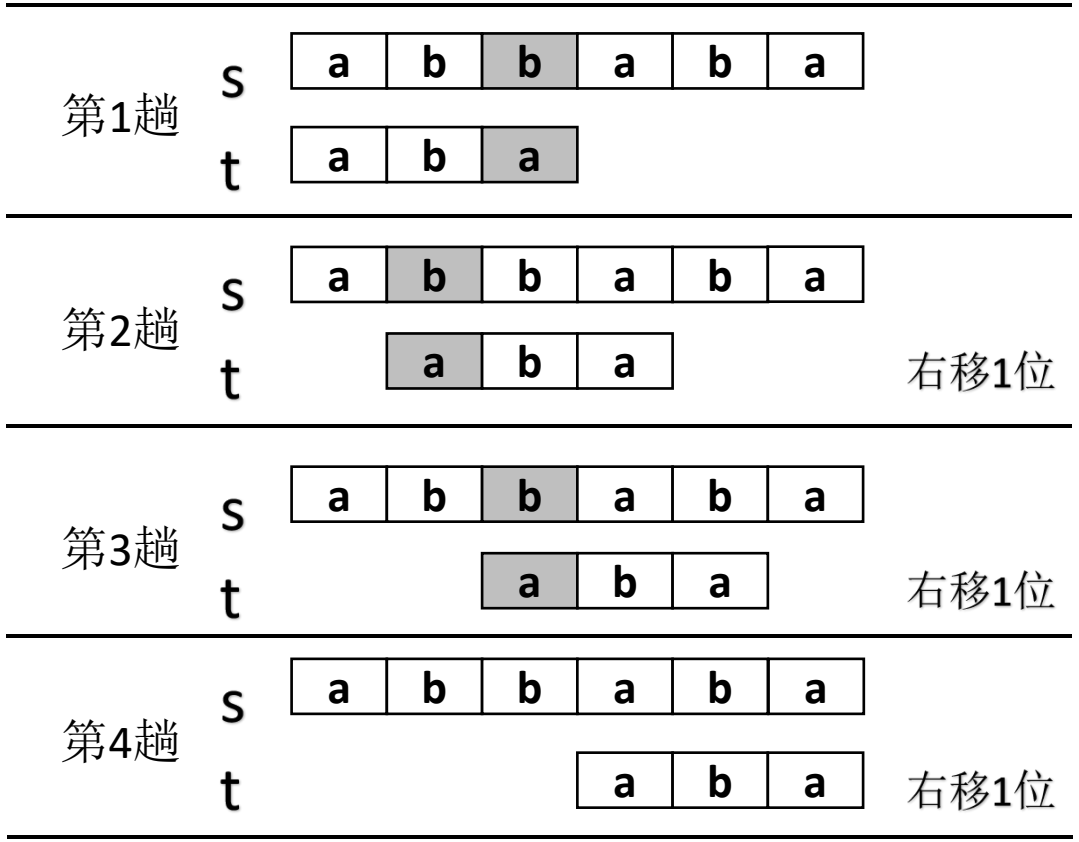
- 又称暴力解法（BF，Brute Force）
  - 枚举每个目标串s与模式串t等长的子串，判断是否匹配
    - 将模式串t的第0位字符与目标串s的第0位字符对齐，然后依次比对每个字符，若都相等，则匹配成功；
    - 若s和t对应位置上的字符不相等，则将t整体后移1位重新从模式串t的第0位开始依次比对。

算法4-11 朴素字符串匹配算法PatternMatchBF(s, t)

```
输入：目标串s与模式串t
输出：返回首个有效位移p，匹配失败则返回NIL
n ← s.length
m ←t.length
for p←0 to n-m do
| for i← 0 to m-1 do
| | if s.data[p+i]≠t.data[i] then
| | | break
| | end
| end
| if i=m then
| | break
| end
end
if p>n-m then
| p ← NIL
end
return p
```

朴素模式匹配算法

例：目标串s="abbaba"，模式串t="aba"



朴素模式匹配算法

- **时间复杂度分析**
  - **最好情况**
    - **子串在字符串s的起始处**
    - **仅需匹配m次，此种情况时间复杂度为O(m)**
  - **最坏情况**
    - **子串不存在，或在字符串s的末尾**
    - **需要移动n-m+1次，每次匹配m次，时间复杂度为$O(nm-m^2)$，即O(nm)**

在实际运行过程中字符串匹配情况复杂多变，朴素模式匹配算法的执行时间通常取上界**O(nm)**。

朴素模式匹配算法

- **字符串匹配是其他部分字符串操作的基础**
  - **如字符串替换：先利用匹配操作找到位置，而后进行删除和插入**

算法 4-12 字符串替换算法Replace(s, sub_s, t)

输入：字符串s，被替换的子串sub_s，替换目标字符串t
输出：替换字符串s中的所有子串sub_s为字符串t后的字符串s

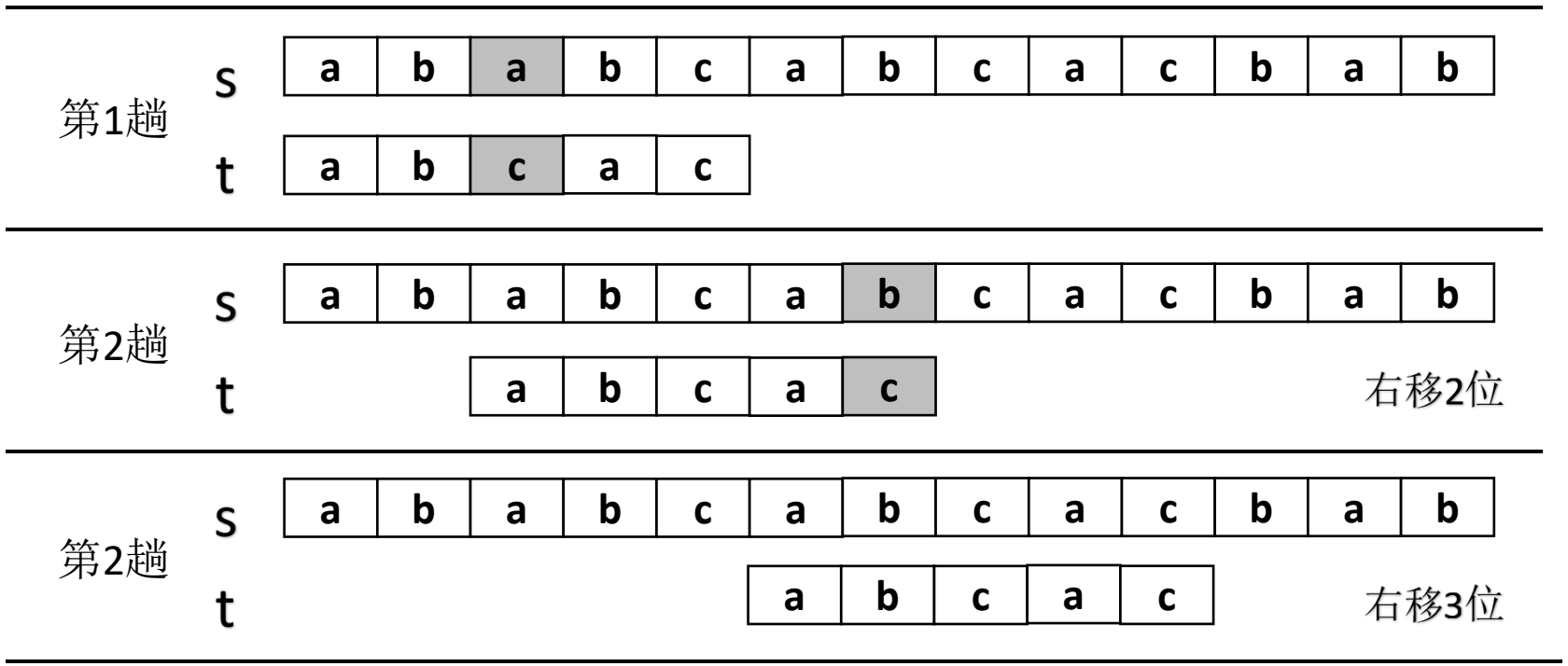
```
len ← sub_s.length
m ← t.length
pos ← 0
while pos ≠ NIL do
  | pos ← PatternMatchBF(s, sub_s)  //从s中找到第一次出现的sub_s
  | if pos ≠ NIL then
  | | StrRemove(s, pos+1, len) //删除sub_s
  | | StrInsert(s, pos+1, t)  //插入t
  | | s.length ← s.length - len + m
  | end
end
```

KMP算法

- **BF算法的缺点**
  - 当模式串失配时将其整体**只向后移动一位**，然后从头开始匹配
  - 能否多移动几位，从而减少比较的次数?
- **KMP算法**
  - 以三位提出者的名字首字母命名
    - D.E.Knuth，J.H.Morris和V.R.Pratt
  - 主要思想
    - 假设BF算法在失配时已经匹配到了字符串的第 j 位，则说明目标串与模式串的前 j-1 位是匹配成功的，**利用已匹配的信息可对BF算法进行优化**

KMP算法

- **例：目标串s="ababcabcacbab"，模式串t="abcac "**



- **可以看出在比较的时候，模式串向后移动了更多的位数，因此能够更快地完成模式匹配**

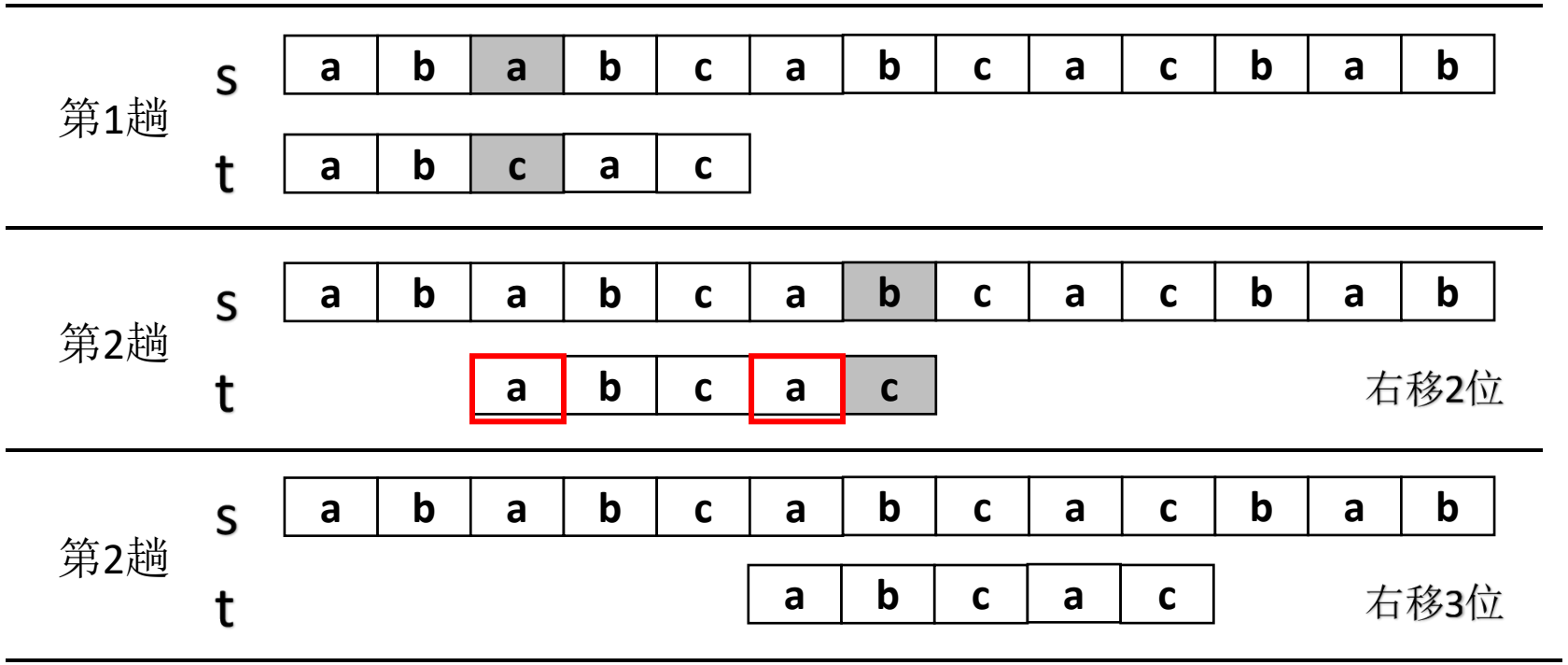
KMP算法

- **思考：** 当出现失配情况时，如何将模式串向右移动正确的位数？
- **基本概念**
  - **前缀**：对字符串s，其前缀表示为s[0, ..., i](0<i≤n-1)
    - 第0位开始，第i位结束的任意子串
    - 字符串的所有前缀构成的集合称为**前缀集合**
  - **后缀**：对字符串s，其后缀可表示为s[i, ..., n-1](0<i≤n-1)
    - 第i位开始，最后一位结束的任意子串
    - 字符串的所有后缀构成的集合称为**后缀集合**。
  - **公共前后缀**：前缀集合与后缀集合中相同的子串
  - **最长公共前后缀**：公共前后缀中长度最长的子串

KMP算法

- 例：字符串” aabaa”
  - 前缀集合
    - {"a","aa","aab","aaba"}
  - 后缀集合
    - {"a","aa","baa","abaa"}
  - 公共前后缀
    - {"a","aa"}
  - 最长公共前后缀
    - “aa”

KMP算法



KMP算法中向右移动的位数由已匹配部分字符串的最长公共前后缀的长度决定。

- **若当前匹配的位置为p，已匹配部分字符串的最长公共前后缀的长度为n，则下一次匹配的起始位置为n**

KMP算法

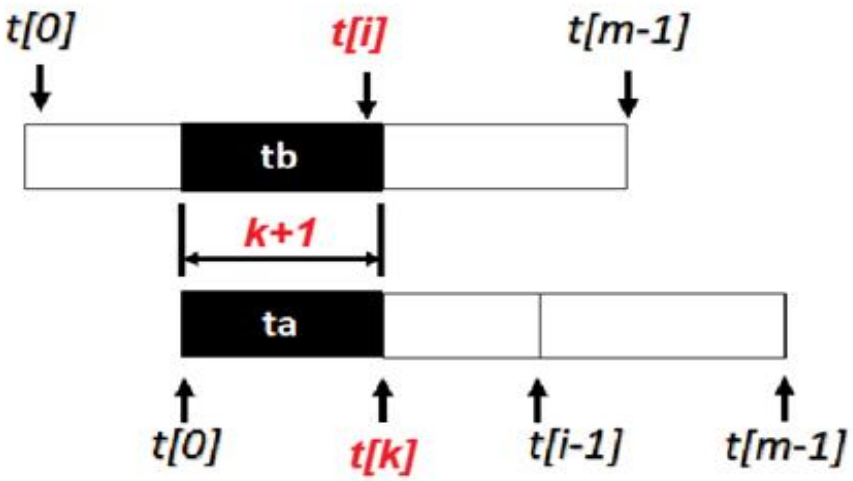
- **每次实时计算已匹配字符串的最长公共前后缀的长度比较耗时**
  - **预先计算待匹配字符串t每个位置处的最长公共前后缀长度**
- **字符串特征向量**
  - **长度为m的字符串t的特征向量是一个m维向量，通常记作next，且用数组形式进行存储，所以特征向量也可非正式地称为next数组**

$$next[i] = \begin{cases} \text{满足}t[0...k] = t[i-k...i]\text{的最大}k(\text{存在}k\text{，且}k < i) \\ -1, \qquad \text{如果这样的}k\text{不存在} \end{cases}$$

  - **next[i]表示的真实含义是字符串t的子串t[0...i]的最长公共前后缀中的前缀最末尾的字符的位置；若最长公共前后缀不存在，则next[i]为-1**
  - **t[0...i]的最长公共前后缀长度为next[i]+1**

KMP算法

- **字符串特征向量的含义**
  - **next[i] = k**
  - **最长公共前后缀的长度为next[i] + 1**



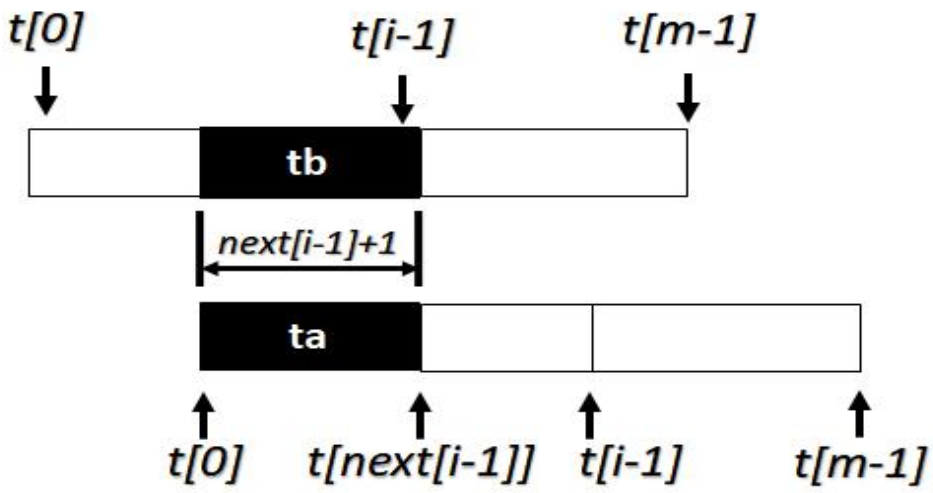
KMP算法

- 例：字符串"abcac"的特征向量为[-1,-1,-1,0,-1]

i	子串 $t[0...i]$	前缀集合	后缀集合	最长公共前后缀	特征向量第i位分量
$i=0$	a	空	空	空	-1
$i=1$	ab	a	b	空	-1
$i=2$	abc	a,ab	c,bc	空	-1
$i=3$	abca	a,ab,abc	a,ca,bca	a	0
$i=4$	abcac	a,ab,abc,abca	c,ac,cac,bcac	空	-1

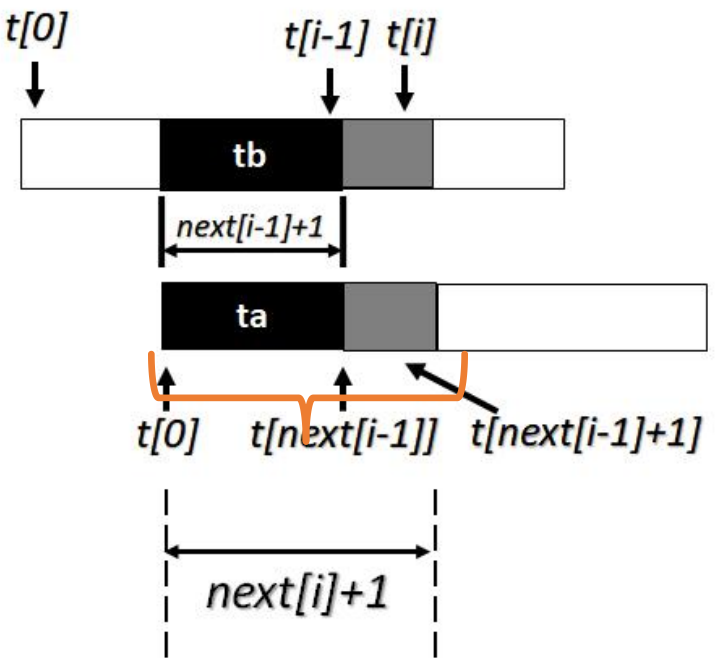
KMP算法

- **如何计算next数组**
  - **若朴素计算next数组，时间复杂度为O(m^3)，不能满足进行快速模式匹配的要求**
- **一种时间复杂度为 O(m)的字符串特征向量算法**
  - **若已知next[i-1]，即ta==tb，如何求next[i]?**



KMP算法

- **一种时间复杂度为 O(m)的字符串特征向量算法**
  - 只需判断tb后面一个字符与ta后面一个字符是否相等
    - 即t[i]与t[next[i-1]+1]是否相等
  - 若相等，则t[0,…,i]的最长公共前后缀+1
    - 即next[i] = next[i-1] + 1
  - 若不相等，需在子串t[0,…,next[i-1]+1]中递归的寻找可能的最长公共前后缀
    - next[i]=next[next[i-1]]+1



KMP算法

- **next数组的求解算法**

$$next[i]=\begin{cases} next[i-1]+1 & t[i]==t[next[i-1]+1] \\ next[next[i-1]]+1 & t[i]==t[next[next[i-1]]+1] \\ \cdots & \cdots \end{cases}$$

$$next[0]=-1$$

- **时间复杂度：O(m)**

算法4-13 求解字符串t的next数组GetNext (t, next)

```
输入：字符串t
输出：字符串t的next数组
m ← t.length
next[0] ← -1
for i←1 to m-1 do    //求出next[1]~next[m-1]
| j ← next[i-1]
| while j ≥ 0 且 t.data[i] ≠ t.data[j+1] do
| | j ← next[j]
| end
| if t.data[i]=t.data[j+1] then
| | next[i] ← j+1
| else
| | next[i] ← -1
| end
end
```

KMP算法

- **KMP算法流程**
  - 在next数组的基础上，在目标串与模式串在模式串的第i位失配时，将模式串右移，从模式串的第next[i-1]+1位开始进行匹配，并不断执行以上步骤直到字符串末尾。
- **时间复杂度：O(n+m)**
  - 算法执行过程中目标串只向右移动，比较的复杂度为O(n)。
  - 计算next数组的复杂度为O(m)，其中m为模式串的长度

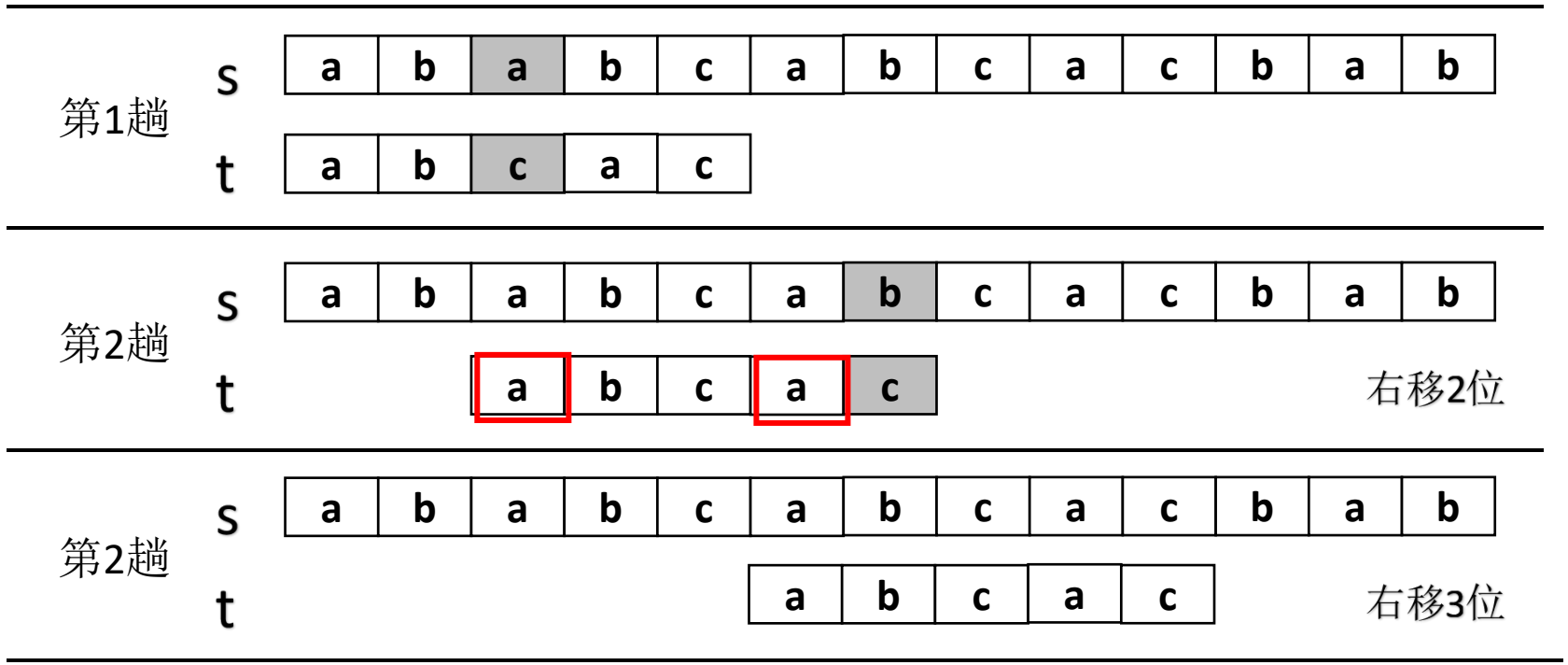
算法4-14 字符串匹配的KMP算法PatternMatchKMP(s, t)

```
输入：目标串s，模式串t
输出：返回首个有效匹配位置p，匹配失败则返回NIL
n ← s.length
m ← t.length
p ← NIL
if n ≥ m then
| GetNext (t, next)
| i ← 0
| j ← 0
| while j<n且i<m do
| | if s.data[j]=t.data[i] then
| | | i ← i+1
| | | j ← j+1
| | else if i>0 then
| | | i ← next[i-1]+1
| | else
| | | j ← j+1
| | end
| end
| if i=m then
| | p ← j-m
| end
end
return p
```

KMP算法

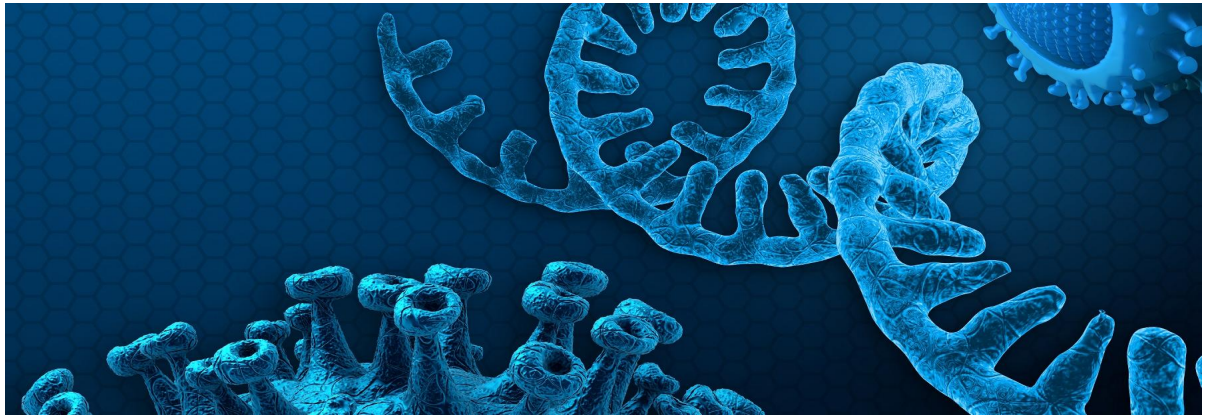
- 例：目标串s="ababcabcacbab"，模式串t="abcac “

i	特征向量第i位分量
$i=0$	-1
$i=1$	-1
$i=2$	-1
$i=3$	0
$i=4$	-1



4.6 应用场景：基因测序

- **示例场景**
  - 生物信息学中已知一串复杂的病毒RNA序列，其规模可以达到百万级别。致病性的RNA序列段规模达到万级，此时科学家需要判断这串病毒的RNA序列中是否包含致病序列段。



- **解决方法：**
  - 将复杂的病毒RNA序列认为是目标串s，致病性的RNA序列段认为是模式串t，使用KMP算法解决模式匹配问题
  - 判断模式串t是否为目标串s的子串，如果是，则可以认定该病毒具有致病性，反之则认为该病毒没有致病性

4.7 小结

字符串和字符类型的线性表的异同点：

字符串：主要是对字符串的整体操作　　**线性表**：是对表中元素的操作。

字符串的主要操作：

字符串插入、字符串删除、字符串截取、字符串连接、字符串比较

字符串的两种实现方法：

顺序存储结构实现，链接存储结构实现

介绍了字符串匹配中的几个重要算法：

朴素算法、KMP算法、BM算法等。

4.7 小结

各种匹配算法的比较：

- 朴素算法通过逐位比较的方式进行匹配，思路最容易理解，算法时间复杂度为O(nm)；
- KMP算法利用利用字符串特征向量加速匹配过程，在任何场景下算法复杂度能够达到稳定的O(n+m),在小字符集模式应用场景下表现尤其突出；